A02b · The One That Does Not Move — Practice A2 · 9618 §19.1

Name: _____

This topic is examined **twice** — described on Paper 3, typed on Paper 4, so this homework has both. **Questions 1–6 are written, on this sheet. Questions 7–10 are code, in `A02b_Practice.java`** (listed at the bottom). One set of numbers across both halves. Throughout, a linked list is stored in a 2D array: **column 0 holds the data, column 1 the index of the next node**, and a pointer of **−1** ends a chain.

1. Read it. A linked list of positive integers is stored in the array **`LinkedList`** below. [3]

Index	Data	NextNode
0	1	3
1	4	0
2	3	5
3	5	−1
4	−1	−1
5	1	1

```
FirstNode  = 2
FirstEmpty = 4
```

(a) Write out the data **in list order**, starting from **`FirstNode`**. [2]

(b) When a traversal reaches node 3, how does it know to stop there? [1]

2. Insert at the front. **`InsertNode(9)`** is called on the list in question 1 — the new value goes at the **front** of the list and both chains are maintained. State each value after the call. [4]

(a) the index of the node that now stores 9

(b) that node's **`NextNode`** value

(c) **`FirstNode`**

(d) **`FirstEmpty`** — and state what that value tells you about the list

3. Complete it. The pseudocode below searches the linked list for a value and reports whether it is there. Fill in the four gaps. [4]

```
INPUT SearchValue
Current ← FirstNode
Found ← FALSE
WHILE ① _____ AND Found = FALSE
    IF ② _____ = SearchValue
        THEN
            Found ← TRUE
        ELSE
            Current ← ③ _____
    ENDIF
ENDWHILE
IF ④ _____
    THEN
        OUTPUT "Found"
    ELSE
        OUTPUT "Not found"
ENDIF
```

4. Remove a middle node. Go back to the table **exactly as printed in question 1** — the insert from question 2 has *not* happened. **`RemoveNode(4)`** is called: it finds the node holding 4, re-points its predecessor, sets the removed node's data to **−1** and returns the node to the free list. Complete the table and both pointers after the call. [4]

Index	Data	NextNode
0	1	3
1		
2	3	5
3	5	−1
4	−1	−1
5	1	

`FirstNode =` _____

`FirstEmpty =` _____

5. The spare nodes. A simpler implementation keeps only **`FirstNode`**, and when a node is deleted it is simply abandoned. Explain how the structure can be improved so that the space in deleted nodes is **reused**. [2]

6. The null that moved. A different program stores a linked list in two 1D arrays whose indices run **from 1 to 8**. State a suitable value for the null pointer in this program, and justify it. [2]

💻 **QUESTIONS 7–10 — ON THE COMPUTER, NOT ON THIS SHEET.** Open **`A02b_Practice.java`**. It contains a short unmarked warm-up and then these questions. The file **tests itself** — and after every operation it checks that *no node has been lost*, which is the mistake that does not crash.

7	**`OutputList()`** — follow the pointers from **`FirstNode`** and print the data in list order	[2]
8	**`InsertNode()`** — insert at the front; FALSE if no free node is left *(save the next-empty index first)*	[6]
9	**`RemoveNode()`** — head, middle and not-found all handled, node handed back	[5]
10	🍦 **`AddToEnd()`** — the **other** insertion policy — optional, not marked	—

To hand in: rename the file to **`A02b_YourName.java`** and submit it. Change nothing inside — the class is not **`public`**, so Java does not mind what the file is called. · **Written 19 marks + code 13 marks = 32.**

🕳 **Want to dig?** Question 4 made you walk the list to find the predecessor, because a node cannot see backwards. There is a version of this structure where every node carries **two** pointers — **`NextNode`** *and* **`PreviousNode`** — and deleting never needs a walk at all. Sketch what **`RemoveNode`** looks like there. Then count what it cost you: how many pointer writes does *insert* need now, and how much of the array is spent on pointers instead of data?

Additional space. If you need more room for any answer, continue here and write the question number in the margin beside it.